Clases

1. Sintaxis básica de `class`
2. Herencia de clase
3. Propiedades y métodos estáticos.
4. Propiedades y métodos privados y protegidos.
5. Ampliación de clases nativas
6. Comprobación de clase: "instanceof"
7. Los Mixins

© 2007—2026 Ilya Kantor

Sintaxis básica de `class`

> *En informática, una clase es una plantilla para la creación de objetos de datos según un modelo predefinido. Las clases se utilizan para representar entidades o conceptos, como los sustantivos en el lenguaje. Cada clase es un modelo que define un conjunto de variables —el estado—, y métodos apropiados para operar con dichos datos —el comportamiento—.*
>
> Wikipedia

En la práctica a menudo necesitamos crear muchos objetos del mismo tipo: usuarios, bienes, lo que sea.

Como ya sabemos del capítulo Constructor, operador "new", `new function` puede ayudar con eso.

Pero en JavaScript moderno hay un constructor más avanzado, "class", que introduce características nuevas muy útiles para la programación orientada a objetos.

La sintaxis "class"

La sintaxis básica es:

```
class MyClass {
  // métodos de clase
  constructor() { ... }
  method1() { ... }
  method2() { ... }
  method3() { ... }
  ...
}
```

Entonces usamos `new MyClass()` para crear un objeto nuevo con todos los métodos listados.

El método `constructor()` es llamado automáticamente por `new`, así podemos inicializar el objeto allí.

Por ejemplo:

```
class User {

  constructor(name) {
    this.name = name;
```

```
  }

  sayHi() {
    alert(this.name);
  }

}

// Uso:
let user = new User("John");
user.sayHi();
```

Cuando se llama a `new User("John")`:

1. Un objeto nuevo es creado.
2. El `constructor` se ejecuta con el argumento dado y lo asigna a `this.name`.

...Entonces podemos llamar a sus métodos, como `user.sayHi()`.

> **No va una coma entre métodos de clase**
>
> Un tropiezo común en desarrolladores principiantes es poner una coma entre los métodos de clase, lo que resulta en un error de sintaxis.
>
> La notación aquí no debe ser confundida con la sintaxis de objeto literal. Dentro de la clase no se requieren comas.

¿Qué es una clase?

Entonces, ¿qué es exactamente `class`? No es una entidad completamente nueva a nivel de lenguaje como uno podría pensar.

Desvelemos la magia y veamos lo que realmente es una clase. Ayudará a entender muchos aspectos complejos.

En JavaScript, una clase es un tipo de función.

Veamos:

```
class User {
  constructor(name) { this.name = name; }
  sayHi() { alert(this.name); }
}

// La prueba: User es una función
alert(typeof User); // function
```

Lo que la construcción `class User {...}` hace realmente es:

1. Crea una función llamada `User`, la que se vuelve el resultado de la declaración de la clase. El código de la función es tomado del método `constructor` (se asume vacío si no se escribe tal método).
2. Almacena los métodos de clase, tales como `sayHi`, en `User.prototype`.

Después de que el objeto `new User` es creado, cuando llamamos a sus métodos estos son tomados del prototipo, tal como se describe en el capítulo

F.prototype. Así el objeto tiene acceso a métodos de clase.

Podemos ilustrar el resultado de la declaración de `class User` como:

Aquí el código para inspeccionarlo:

```
class User {
  constructor(name) { this.name = name; }
  sayHi() { alert(this.name); }
}

// una clase es una función
alert(typeof User); // function

// ...o, más precisamente, el método constructor
alert(User === User.prototype.constructor); // true

// Los métodos están en User.prototype, por ejemplo:
alert(User.prototype.sayHi); // el código del método s

// Hay exactamente dos métodos en el prototipo
alert(Object.getOwnPropertyNames(User.prototype)); //
```

No es solamente azúcar sintáctica

A veces se dice que `class` es "azúcar sintáctica" (sintaxis que es diseñada para una lectura más fácil, pero que no introduce nada nuevo), porque en realidad podemos declarar lo mismo sin la palabra clave `class` en absoluto:

```
// reescribiendo la clase User puramente con funciones

// 1. Crear la función constructor
function User(name) {
  this.name = name;
}
// un prototipo de función tiene la propiedad "constru
// así que no necesitamos crearla

// 2. Agregar el método al prototipo
User.prototype.sayHi = function() {
  alert(this.name);
};

// Uso:
let user = new User("John");
user.sayHi();
```

El resultado de esta definición es el mismo. Así, efectivamente hay razones para que `class` sea considerada azúcar sintáctica para definir un constructor junto

con sus métodos de prototipo.

Aún así hay diferencias importantes.

1. Primero, una función creada por `class` es etiquetada por una propiedad interna especial `[[IsClassConstructor]]:true`. Entones no es exactamente lo mismo que crearla manualmente.

   El lenguaje verifica esa propiedad en varios lugares. Por ejemplo, a diferencia de las funciones regulares, esta debe ser llamada con `new`:

   ```
   1 class User {
   2   constructor() {}
   3 }
   4
   5 alert(typeof User); // function
   6 User(); // Error: El constructor de clase User no pu
   ```

   Además una representación string de un constructor de clase en la mayoría de los motores JavaScript comienzan con "class..."

   ```
   1 class User {
   2   constructor() {}
   3 }
   4
   5 alert(User); // class User { ... }
   ```

   Hay otras diferencias que veremos pronto.

2. Los métodos de clase no son enumerables. La definición de clase establece la bandera `enumerable` a `false` para todos los métodos en `"prototype"`.

   Esto es bueno porque si hacemos `for..in` a un objeto usualmente no queremos sus métodos de clase.

3. Las clases siempre asumen `use strict`. Todo el código dentro del constructor de clase está automáticamente en modo estricto.

Además la sintaxis de `class` brinda muchas otras características que exploraremos luego.

Expresión de clases

Al igual que las funciones, las clases pueden ser definidas dentro de otra expresión, pasadas, devueltas, asignadas, etc.

Aquí hay un ejemplo de una expresión de clase:

```
1 let User = class {
2   sayHi() {
3     alert("Hello");
4   }
5 };
```

Al igual que las expresiones de función, las expresiones de clase pueden tener un nombre.

Si una expresión de clase tiene un nombre, este es visible solamente dentro de la clase.

```
// Expresiones de clase con nombre
// ("Named Class Expression" no figura así en la espec
let User = class MyClass {
  sayHi() {
    alert(MyClass); // El nombre de MyClass solo es vi
  }
};

new User().sayHi(); // Funciona, muestra la definición

alert(MyClass); // error, el nombre de MyClass no es v
```

Podemos inclusive crear clases dinámicamente "a pedido", como esto:

```
function makeClass(phrase) {
  // declara una clase y la devuelve
  return class {
    sayHi() {
      alert(phrase);
    }
  };
}

// Crea una nueva clase
let User = makeClass("Hello");

new User().sayHi(); // Hello
```

Getters/setters

Al igual que los objetos literales, las clases pueden incluir getters/setters, propiedades calculadas, etc.

Aquí hay un ejemplo de `user.name`, implementado usando `get/set`:

```
class User {

  constructor(name) {
    // invoca el setter
    this.name = name;
  }

  get name() {
    return this._name;
  }

  set name(value) {
    if (value.length < 4) {
      alert("Nombre demasiado corto.");
```

```
15        return;
16      }
17      this._name = value;
18    }
19
20  }
21
22  let user = new User("John");
23  alert(user.name); // John
24
25  user = new User(""); // Nombre demasiado corto.
```

Técnicamente, la declaración de clase funciona creando getters y setters en `User.prototype`.

Nombres calculados [...]

Aquí hay un ejemplo con un nombre de método calculado usando corchetes `[...]`:

```
1  class User {
2
3    ['say' + 'Hi']() {
4      alert("Hello");
5    }
6
7  }
8
9  new User().sayHi();
```

Es una característica fácil de recordar porque se asemeja a la de los objetos literales.

Campos de clase (Class fields)

> **Los navegadores viejos pueden necesitar polyfill**
>
> Los campos de clase son un agregado reciente al lenguaje.

Antes, nuestras clases tenían solamente métodos.

"Campos de clase" es una sintaxis que nos permite agregar una propiedad cualquiera.

Por ejemplo, agreguemos la propiedad `name` a la clase `User`:

```
1  class User {
2    name = "John";
3
4    sayHi() {
5      alert(`Hello, ${this.name}!`);
6    }
7  }
8
9
```

```
new User().sayHi(); // Hello, John!
```

Así, simplemente escribimos " = " en la declaración, y eso es todo.

La diferencia importante de las propiedades definidas como "campos de clase" es que estas son establecidas en los objetos individuales, no compartidas en `User.prototype`:

```
class User {
  name = "John";
}

let user = new User();
alert(user.name); // John
alert(User.prototype.name); // undefined
```

También podemos asignar valores usando expresiones más complejas y llamados a función:

```
class User {
  name = prompt("Name, please?", "John");
}

let user = new User();
alert(user.name); // John
```

Vinculación de métodos (binding) usando campos de clase

Como se demostró en el capítulo Función bind: vinculación de funciones, las funciones en JavaScript tienen un `this` dinámico. Este depende del contexto del llamado.

Entonces si un método de objeto es pasado y llamado en otro contexto, `this` ya no será una referencia a su objeto.

Por ejemplo, este código mostrará `undefined`:

```
class Button {
  constructor(value) {
    this.value = value;
  }

  click() {
    alert(this.value);
  }
}

let button = new Button("hello");

setTimeout(button.click, 1000); // undefined
```

Este problema es denominado "pérdida de `this`".

Hay dos enfoques para solucionarlo, como se discute en el capítulo Función bind: vinculación de funciones:

1. Pasar un contenedor o wrapper-function como: `setTimeout(() => button.click(), 1000)`.
2. Vincular el método al objeto, por ejemplo en el constructor.

Los campos de clase brindan otra sintaxis, bastante elegante:

```
class Button {
  constructor(value) {
    this.value = value;
  }
  click = () => {
    alert(this.value);
  }
}

let button = new Button("hello");

setTimeout(button.click, 1000); // hello
```

Un campo de clase `click = () => {...}` es creado para cada objeto. Hay una función para cada objeto `Button`, con `this` dentro referenciando ese objeto. Podemos pasar `button.click` a cualquier lado y el valor de `this` siempre será el correcto.

Esto es especialmente práctico, en el ambiente de los navegadores, para los "event listeners".

Resumen

La sintaxis básica de clase se ve así:

```
class MyClass {
  prop = value; // propiedad

  constructor(...) { // constructor
    // ...
  }

  method(...) {} // método

  get something(...) {} // método getter
  set something(...) {} // método setter

  [Symbol.iterator]() {} // método con nombre calculad
  // ...
}
```

`MyClass` es técnicamente una función (la que proveemos como `constructor`), mientras que los métodos, getters y setters son escritos en `MyClass.prototype`.

En los siguientes capítulos aprenderemos más acerca de clases, incluyendo herencia y otras características.

Tareas

Reescribir como class

importancia: 5

La clase `Clock` (ver en el sandbox) está escrita en estilo funcional. Reescríbela en sintaxis de clase.

P.D. El reloj anda en la consola, ábrela para verlo.

Abrir un entorno controlado para la tarea.

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

Herencia de clase

La herencia de clase es el modo para que una clase extienda a otra.

De esta manera podemos añadir nueva funcionalidad a la ya existente.

La palabra clave “extends”

Digamos que tenemos la clase `Animal`:

```
class Animal {
  constructor(name) {
    this.speed = 0;
    this.name = name;
  }
  run(speed) {
    this.speed = speed;
    alert(`${this.name} corre a una velocidad de ${thi
  }
  stop() {
    this.speed = 0;
    alert(`${this.name} se queda quieto.`);
  }
}

let animal = new Animal("Mi animal");
```

Así es como podemos representar gráficamente el objeto `animal` y la clase `Animal`:

...Y nos gustaría crear otra clase `Rabbit`.

Como los conejos son animales, la clase ‘Rabbit’ debería basarse en ‘Animal’ y así tener acceso a métodos animales, para que los conejos puedan hacer lo que los animales “genéricos” pueden hacer.

La sintaxis para extender otra clase es: `class Hijo extends Padre`.

Construyamos la clase `Rabbit` que herede de `Animal`:

```
class Rabbit extends Animal {
  hide() {
    alert(`¡${this.name} se esconde!`);
  }
}

let rabbit = new Rabbit("Conejo Blanco");

rabbit.run(5); // Conejo Blanco corre a una velocidad
rabbit.hide(); // ¡Conejo Blanco se esconde!
```

Los objetos de la clase `Rabbit` tienen acceso a los métodos de `Rabbit`, como `rabbit.hide()`, y también a los métodos `Animal`, como `rabbit.run()`.

Internamente, la palabra clave `extends` funciona con la buena mecánica de prototipo: establece `Rabbit.prototype.[[Prototype]]` a `Animal.prototype`. Entonces, si no se encuentra un método en `Rabbit.prototype`, JavaScript lo toma de `Animal.prototype`.

Por ejemplo, para encontrar el método `rabbit.run`, el motor revisa (en la imagen, de abajo hacia arriba):

1. El objeto `rabbit`: no tiene el método `run`.
2. Su prototipo, que es `Rabbit.prototype`: tiene el método `hide`, pero no el método `run`.
3. Su prototipo, que es `Animal.prototype` (debido a `extends`): Este finalmente tiene el método `run`.

Como podemos recordar del capítulo Prototipos nativos, JavaScript usa la misma herencia prototípica para los objetos incorporados. Por ejemplo, `Date.prototype.[[Prototype]]` es `Object.prototype`. Es por esto que "Date" tiene acceso a métodos de objeto genéricos.

Cualquier expresión está permitida después de `extends`

La sintaxis de clase permite especificar no solo una clase, sino cualquier expresión después de `extends`.

Por ejemplo, una llamada a función que genera la clase padre:

```
function f(phrase) {
  return class {
    sayHi() { alert(phrase); }
  };
}

class User extends f("Hola") {}

new User().sayHi(); // Hola
```

Observa que `class User` hereda del resultado de `f("Hola")`.

Eso puede ser útil para patrones de programación avanzados cuando usamos funciones para generar clases dependiendo de muchas condiciones y podamos heredar de ellas.

Sobrescribir un método

Ahora avancemos y sobrescribamos un método. Por defecto, todos los métodos que no están especificados en la clase `Rabbit` se toman directamente "tal cual" de la clase `Animal`.

Pero Si especificamos nuestro propio método `stop()` en `Rabbit`, es el que se utilizará en su lugar:

```
class Rabbit extends Animal {
  stop() {
    // ...esto se usará para rabbit.stop()
    // en lugar de stop() de la clase Animal
  }
}
```

Sin embargo, no siempre queremos reemplazar totalmente un método padre sino construir sobre él, modificarlo o ampliar su funcionalidad. Hacemos algo con nuestro método, pero queremos llamar al método padre antes, después o durante el proceso.

Las clases proporcionan la palabra clave `"super"` para eso.

- `super.metodo(...)` llama un método padre.
- `super(...)` llama un constructor padre (solo dentro de nuestro constructor).

Por ejemplo, hagamos que nuestro conejo se oculte automáticamente cuando se detenga:

```
class Animal {

  constructor(name) {
    this.speed = 0;
    this.name = name;
  }

  run(speed) {
    this.speed = speed;
```

```
    alert(`${this.name} corre a una velocidad de ${thi
  }

  stop() {
    this.speed = 0;
    alert(`${this.name} se queda quieto.`);
  }

}

class Rabbit extends Animal {
  hide() {
    alert(`¡${this.name} se esconde!`);
  }

  stop() {
    super.stop(); // llama el stop padre
    this.hide(); // y luego hide
  }
}

let rabbit = new Rabbit("Conejo Blanco");

rabbit.run(5); // Conejo Blanco corre a una velocidad
```

Ahora `Rabbit` tiene el método `stop` que llama al padre `super.stop()` en el proceso.

Las funciones de flecha no tienen `super`

Como se mencionó en el capítulo Funciones de flecha revisadas, las funciones de flecha no tienen `super`.

Si se lo accede, lo toma de la función externa. Por ejemplo:

```
class Rabbit extends Animal {
  stop() {
    setTimeout(() => super.stop(), 1000); // lla
  }
}
```

El método `super` en la función de flecha es el mismo que en `stop()`, y funciona según lo previsto. Si aquí especificáramos una función "regular", habría un error:

```
// super inesperado
setTimeout(function() { super.stop() }, 1000);
```

Sobrescribir un constructor

Con los constructores se pone un poco complicado.

Hasta ahora, `Rabbit` no tenía su propio `constructor`.

De acuerdo con la especificación, si una clase extiende otra clase y no tiene `constructor`, se genera el siguiente `constructor` "vacío":

```
class Rabbit extends Animal {
  // es generado por extender la clase sin constructor
  constructor(...args) {
    super(...args);
  }
}
```

Como podemos ver, básicamente llama al `constructor` padre pasándole todos los argumentos. Esto sucede si no escribimos un constructor propio.

Ahora agreguemos un constructor personalizado a `Rabbit`. Especificará `earLength` además de `name`:

```
class Animal {
  constructor(name) {
    this.speed = 0;
    this.name = name;
  }
  // ...
}

class Rabbit extends Animal {

  constructor(name, earLength) {
    this.speed = 0;
    this.name = name;
    this.earLength = earLength;
  }

  // ...
}

// No funciona!
let rabbit = new Rabbit("Conejo Blanco", 10); // Error
```

¡Vaya! Tenemos un error. Ahora no podemos crear conejos. ¿Qué salió mal?

La respuesta corta es:

- **Los constructores en las clases heredadas deben llamar a `super(...)`, y (¡!) hacerlo antes de usar `this`.**

...¿Pero por qué? ¿Qué está pasando aquí? De hecho, el requisito parece extraño.

Por supuesto, hay una explicación. Vamos a entrar en detalles, para que realmente entiendas lo que está pasando.

En JavaScript, hay una distinción entre una función constructora de una clase heredera (llamada "constructor derivado") y otras funciones. Un constructor derivado tiene una propiedad interna especial `[[ConstructorKind]]:"derived"`. Esa es una etiqueta interna especial.

Esa etiqueta afecta su comportamiento con `new`.

- Cuando una función regular se ejecuta con `new`, crea un objeto vacío y lo asigna a `this`.
- Pero cuando se ejecuta un constructor derivado, no hace esto. Espera que el constructor padre haga este trabajo.

Entonces un constructor derivado debe llamar a `super` para ejecutar su constructor padre (base), de lo contrario no se creará el objeto para `this`. Y obtendremos un error.

Para que el constructor `Rabbit` funcione, necesita llamar a `super()` antes de usar `this`, como aquí:

```
class Animal {

  constructor(name) {
    this.speed = 0;
    this.name = name;
  }

  // ...
}

class Rabbit extends Animal {

  constructor(name, earLength) {
    super(name);
    this.earLength = earLength;
  }

  // ...
}

// todo bien ahora
let rabbit = new Rabbit("Conejo Blanco", 10);
alert(rabbit.name); // Conejo Blanco
alert(rabbit.earLength); // 10
```

Sobrescribiendo campos de clase: una nota con trampa

> **Nota avanzada**
>
> Esta nota asume que tienes cierta experiencia con clases, quizás en otros lenguajes de programación.
>
> Brinda una visión más profunda al lenguaje y también explica el comportamiento que podría causar errores (pero no muy a menudo).
>
> Si lo encuentras difícil de entender, simplemente sigue adelante, continúa leyendo y vuelve aquí más adelante.

Podemos sobrescribir no solo métodos, sino también los campos de la clase.

Pero hay un comportamiento peculiar cuando accedemos a los campos sobrescritos en el constructor padre, muy diferente a de la mayoría de los demás lenguajes de programación.

Considera este ejemplo:

```
class Animal {
  name = 'animal';

constructor() {
    alert(this.name); // (*)
  }
}

class Rabbit extends Animal {
  name = 'rabbit';
}

new Animal(); // animal
new Rabbit(); // animal
```

Aquí, la clase `Rabbit` extiende `Animal` y sobrescribe el campo `name` con un valor propio.

`Rabbit` no tiene su propio constructor, entonces es llamado el de `Animal`.

Lo interesante es que en ambos casos: `new Animal()` y `new Rabbit()`, el `alert` en la línea `(*)` muestra `animal`.

En otras palabras, el constructor padre siempre usa el valor de su propio campo de clase, no el sobrescrito.

¿Qué es lo extraño de esto?

Si esto aún no está claro, comparáralo con lo que ocurre con los métodos.

Aquí está el mismo código, pero en lugar del campo `this.name` llamamos el método `this.showName()`:

```
class Animal {
  showName() {  // en vez de this.name = 'animal'
    alert('animal');
  }

  constructor() {
    this.showName(); // en vez de alert(this.name);
  }
}

class Rabbit extends Animal {
  showName() {
    alert('rabbit');
  }
}

new Animal(); // animal
new Rabbit(); // rabbit
```

Observa que ahora la salida es diferente.

Y es lo que esperamos naturalmente. Cuando el constructor padre es llamado en la clase derivada, usa el método sobrescrito.

...Pero con los campos esto no es así. Como dijimos antes, el constructor padre siempre utiliza el campo padre.

¿Por que existe la diferencia?

Bien, la razón está en el orden de inicialización, El campo de clase es inicializado:

- Antes del constructor para la clase de base (que no extiende nada),
- Inmediatamente después de `super()` para la clase derivada.

En nuestro caso, `Rabbit` es la clase derivada. No hay `constructor()` en ella. Como establecimos previamente, es lo mismo que si hubiera un constructor vacío con solamente `super(...args)`.

Entonces, `new Rabbit()` llama a `super()` y se ejecuta el constructor padre, y (por la regla de la clase derivada) solamente después de que sus campos de clase sean inicializados. En el momento de la ejecución del constructor padre, todavía no existen los campos de clase de `Rabbit`, por ello los campos de `Animal` son los usados.

Esta sutil diferencia entre campos y métodos es particular de JavaScript

Afortunadamente este comportamiento solo se revela si los campos sobrescritos son usados en el constructor padre. En tal caso puede ser difícil entender qué es lo que está pasando, por ello lo explicamos aquí.

Si esto se vuelve un problema, uno puede corregirlo usando métodos o getters/setters en lugar de campos.

Super: internamente, [[HomeObject]]

> **Información avanzada**
>
> Si está leyendo el tutorial por primera vez, esta sección puede omitirse.
>
> Esta sección trata de los mecanismos internos detrás de la herencia y el método `super`.

Vamos a profundizar un poco más el tema de `super`. Veremos algunas cosas interesantes en el camino.

En primer lugar, de todo lo que hemos aprendido hasta ahora, ¡es imposible que `super` funcione en absoluto!

Entonces, preguntémonos: ¿cómo debería funcionar técnicamente? Cuando se ejecuta un método de objeto, obtiene el objeto actual como `this`. Si llamamos a `super.method()` entonces, el motor necesita obtener el `method` del prototipo del objeto actual. ¿Pero cómo?

La tarea puede parecer simple, pero no lo es. El motor conoce el objeto actual `this`, por lo que podría obtener el `method` padre como `this.__proto__.method`. Desafortunadamente, una solución tan "ingenua" no funcionará.

Demostremos el problema. Sin clases, usando objetos puros por simplicidad.

Puedes omitir esta parte e ir a la subsección `[[HomeObject]]` si no deseas conocer los detalles. Eso no hará daño. O sigue leyendo si estás interesado en comprender las cosas en profundidad.

En el siguiente ejemplo, se hace la asignación `rabbit.__proto__ = animal`. Ahora intentemos: en `rabbit.eat()` llamaremos a `animal.eat()`, usando `this.__proto__`:

```
let animal = {
  name: "Animal",
  eat() {
    alert(`${this.name} come.`);
  }
};

let rabbit = {
  __proto__: animal,
  name: "Conejo",
  eat() {
    // así es como supuestamente podría funcionar supe
    this.__proto__.eat.call(this); // (*)
  }
};

rabbit.eat(); // Conejo come.
```

En la línea `(*)` tomamos `eat` del prototipo (`animal`) y lo llamamos en el contexto del objeto actual. Tenga en cuenta que `.call(this)` es importante aquí, porque un simple `this.__proto__.eat()` ejecutaría al padre `eat` en el contexto del prototipo, no del objeto actual.

Y en el código anterior, funciona según lo previsto: tenemos el `alert` correcto.

Ahora agreguemos un objeto más a la cadena. Veremos cómo se rompen las cosas:

```
let animal = {
  name: "Animal",
  eat() {
    alert(`${this.name} come.`);
  }
};

let rabbit = {
  __proto__: animal,
  eat() {
    // ...rebota al estilo de conejo y llama al método
    this.__proto__.eat.call(this); // (*)
  }
};

let longEar = {
  __proto__: rabbit,
  eat() {
    // ...haz algo con orejas largas y llama al método
    this.__proto__.eat.call(this); // (**)
  }
};

longEar.eat(); // Error: Se excedió el número máximo d
```

¡El código ya no funciona! Podemos ver el error al intentar llamar a `longEar.eat()`.

Puede que no sea tan obvio, pero si depuramos la llamada `longEar.eat()`, podremos ver por qué. En ambas líneas `(*)` y `(**)` el valor de `this` es el objeto actual (`longEar`). Eso es esencial: todos los métodos de objeto obtienen el objeto actual como `this`, no un prototipo o algo así.

Entonces, en ambas líneas `(*)` y `(**)` el valor de `this.__proto__` es exactamente el mismo: `rabbit`. Ambos llaman a `rabbit.eat` sin subir la cadena en el bucle sin fin.

Aquí está la imagen de lo que sucede:

1. Dentro de `longEar.eat()`, la línea `(**)` llama a `rabbit.eat` proporcionándole `this=longEar`.

```
// dentro de longEar.eat() tenemos this = longEar
this.__proto__.eat.call(this) // (**)
// se convierte en
```

```
longEar.__proto__.eat.call(this)
// es decir
rabbit.eat.call(this);
```

2. Luego, en la línea `(*)` de `rabbit.eat`, queremos pasar la llamada aún más arriba en la cadena; pero como `this=longEar`, entonces `this.__proto__.eat` ¡es nuevamente `rabbit.eat`!

```
// dentro de rabbit.eat () también tenemos this = lo
this.__proto__.eat.call(this) // (*)
// se convierte en
longEar.__proto__.eat.call(this)
// o (de nuevo)
rabbit.eat.call(this);
```

3. ...Entonces `rabbit.eat` se llama a sí mismo en el bucle sin fin, porque no puede ascender más.

El problema no se puede resolver usando solamente `this`.

[[HomeObject]]

Para proporcionar la solución, JavaScript agrega una propiedad interna especial para las funciones: `[[HomeObject]]`.

Cuando una función se especifica como un método de clase u objeto, su propiedad `[[HomeObject]]` se convierte en ese objeto.

Entonces `super` lo usa para resolver el problema del prototipo padre y sus métodos.

Veamos cómo funciona, primero con objetos simples:

```
let animal = {
  name: "Animal",
  eat() {         // animal.eat.[[HomeObject]] == anim
    alert(`${this.name} come.`);
  }
};

let rabbit = {
  __proto__: animal,
  name: "Conejo",
  eat() {         // rabbit.eat.[[HomeObject]] == rabb
    super.eat();
  }
};

let longEar = {
  __proto__: rabbit,
  name: "Oreja Larga",
  eat() {         // longEar.eat.[[HomeObject]] == lon
    super.eat();
  }
};

```

```
24
25 // funciona correctamente
   longEar.eat(); // Oreja larga come
```

Funciona según lo previsto, debido a la mecánica de `[[HomeObject]]`. Un método, como `longEar.eat`, conoce su `[[HomeObject]]` y toma el método padre de su prototipo. Sin el uso de `this`.

Los métodos no son “libres”

Como aprendimos antes, generalmente las funciones son “libres”, es decir que no están vinculadas a objetos en JavaScript. Esto es para que puedan copiarse entre objetos y llamarse con otro 'this`.

La existencia misma de `[[HomeObject]]` viola ese principio, porque los métodos recuerdan sus objetos. `[[HomeObject]]` no se puede cambiar, por lo que este vínculo es para siempre.

El único lugar en el lenguaje donde se usa `[[HomeObject]]` es en `super`. Si un método no usa `super`, entonces todavía podemos considerarlo “libre” y copiarlo entre objetos. Pero con `super` las cosas pueden salir mal.

Aquí está la demostración de un resultado incorrecto de `super` después de copiarlo:

```
1  let animal = {
2    sayHi() {
3      alert(`Soy un animal`);
4    }
5  };
6
7  // rabbit hereda de animal
8  let rabbit = {
9    __proto__: animal,
10   sayHi() {
11     super.sayHi();
12   }
13 };
14
15 let plant = {
16   sayHi() {
17     alert("Soy una planta");
18   }
19 };
20
21 // tree hereda de plant
22 let tree = {
23   __proto__: plant,
24   sayHi: rabbit.sayHi // (*)
25 };
26
27 tree.sayHi();  // Soy un animal (?!?)
```

Una llamada a `tree.sayHi()` muestra “Soy un animal”. Definitivamente mal.

La razón es simple:

- En la línea `(*)`, el método `tree.sayHi` se copió de `rabbit`. ¿Quizás solo queríamos evitar la duplicación de código?

- Su `[[HomeObject]]` es `rabbit`, ya que fue creado en `rabbit`. No hay forma de cambiar `[[HomeObject]]`.
- El código de `tree.sayHi()` tiene dentro a `super.sayHi()`. Sube desde 'rabbit' y toma el método de 'animal'.

Aquí está el diagrama de lo que sucede:

Métodos, no propiedades de función

`[[HomeObject]]` se define para métodos tanto en clases como en objetos simples. Pero para los objetos, los métodos deben especificarse exactamente como `method()`, no como `"method: function()"`.

La diferencia puede no ser esencial para nosotros, pero es importante para JavaScript.

En el siguiente ejemplo, se utiliza una sintaxis sin método para la comparación. La propiedad `[[HomeObject]]` no está establecida y la herencia no funciona:

```
let animal = {
  eat: function() { // escrito así intencionalmente en
    // ...
  }
};

let rabbit = {
  __proto__: animal,
  eat: function() {
    super.eat();
  }
};

rabbit.eat();  // Error al llamar a super (porque no h
```

Resumen

1. Para extender una clase: `class Hijo extends Padre`: – Eso significa que `Hijo.prototype.__proto__` será `Padre.prototype`, por lo que los métodos se heredan.
2. Al sobrescribir un constructor: – Debemos llamar al constructor del padre `super()` en el constructor de `Hijo` antes de usar `this`.
3. Al sobrescribir otro método: – Podemos usar `super.method()` en un método `Hijo` para llamar al método `Padre`.
4. Características internas: – Los métodos recuerdan su clase/objeto en la propiedad interna `[[HomeObject]]`. Así es como `super` resuelve los

métodos padres. – Por lo tanto, no es seguro copiar un método con `super` de un objeto a otro.

También:

- Las funciones de flecha no tienen su propio `this` o `super`, por lo que se ajustan de manera transparente al contexto circundante.

Tareas

Error al crear una instancia

importancia: 5

Aquí está el código de la clase `Rabbit` que extiende a `Animal`.

Desafortunadamente, los objetos `Rabbit` no se pueden crear. ¿Que pasa? Arréglalo.

```
class Animal {

  constructor(name) {
    this.name = name;
  }

}

class Rabbit extends Animal {
  constructor(name) {
    this.name = name;
    this.created = Date.now();
  }
}

let rabbit = new Rabbit("Conejo Blanco"); // Error: th
alert(rabbit.name);
```

solución

Reloj extendido

importancia: 5

Tenemos una clase 'Clock'. Por ahora, muestra la hora cada segundo.

```
class Clock {
  constructor({ template }) {
    this.template = template;
  }

  render() {
    let date = new Date();

    let hours = date.getHours();
    if (hours < 10) hours = '0' + hours;
```

```
11
12      let mins = date.getMinutes();
13      if (mins < 10) mins = '0' + mins;
14
15      let secs = date.getSeconds();
16      if (secs < 10) secs = '0' + secs;
17
18      let output = this.template
19        .replace('h', hours)
20        .replace('m', mins)
21        .replace('s', secs);
22
23      console.log(output);
24    }
25
26    stop() {
27      clearInterval(this.timer);
28    }
29
30    start() {
31      this.render();
32      this.timer = setInterval(() => this.render(), 1000
33    }
```

Crea una nueva clase `ExtendedClock` que herede de `Clock` y agrega el parámetro `precision` : este es el número de `milisegundos` entre "tics". Debe ser `1000` (1 segundo) por defecto.

- Tu código debe estar en el archivo `extended-clock.js`
- No modifiques el `clock.js` original. Extiéndelo.

Abrir un entorno controlado para la tarea.

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

acerca del proyecto
contáctenos

 6 de mayo de 2022

Propiedades y métodos estáticos.

También podemos asignar un método a la clase como un todo. Dichos métodos se llaman *estáticos*.

En la declaración de una clase, se preceden por la palabra clave `static`:

```
class User {
  static staticMethod() {
    alert(this === User);
  }
}

User.staticMethod(); // verdadero
```

Eso realmente hace lo mismo que asignarlo como una propiedad directamente:

```
class User { }

User.staticMethod = function() {
  alert(this === User);
};

User.staticMethod(); // verdadero
```

El valor de `this` en la llamada `User.staticMethod()` es el mismo constructor de clase `User` (la regla "objeto antes de punto").

Por lo general, los métodos estáticos se utilizan para implementar funciones que pertenecen a la clase como un todo, no a un objeto particular de la misma.

Por ejemplo, tenemos objetos `Article` y necesitamos una función para compararlos.

Una solución natural sería agregar el método `Article.compare`:

```
class Article {
  constructor(title, date) {
    this.title = title;
    this.date = date;
  }

  static compare(articleA, articleB) {
    return articleA.date - articleB.date;
  }
}

// uso
```

```
let articles = [
  new Article("HTML", new Date(2019, 1, 1)),
  new Article("CSS", new Date(2019, 0, 1)),
  new Article("JavaScript", new Date(2019, 11, 1))
];

articles.sort(Article.compare);

alert( articles[0].title ); // CSS
```

Aquí el método `Article.compare` se encuentra "encima" de los artículos, como un medio para compararlos. No es el método de un artículo sino de toda la clase.

Otro ejemplo sería un método llamado "factory".

Digamos que necesitamos múltiples formas de crear un artículo:

1. Crearlo por parámetros dados (`title`, `date` etc.).
2. Crear un artículo vacío con la fecha de hoy.
3. ... o cualquier otra manera.

La primera forma puede ser implementada por el constructor. Y para la segunda podemos hacer un método estático de la clase.

Tal como `Article.createTodays()` aquí:

```
class Article {
  constructor(title, date) {
    this.title = title;
    this.date = date;
  }

  static createTodays() {
    // recuerda, this = Article
    return new this("Resumen de hoy", new Date());
  }
}

let article = Article.createTodays();

alert( article.title ); // Resumen de hoy
```

Ahora, cada vez que necesitamos crear un resumen de hoy, podemos llamar a `Article.createTodays()`. Una vez más, ese no es el método de un objeto artículo, sino el método de toda la clase.

Los métodos estáticos también se utilizan en clases relacionadas con base de datos para buscar/guardar/eliminar entradas de la misma, como esta:

```
// suponiendo que Article es una clase especial para g
// método estático para eliminar el artículo por id:
Article.remove({id: 12345});
```

Los métodos estáticos no están disponibles para objetos individuales

Los métodos estáticos son llamados sobre las clases, no sobre los objetos individuales.

Por ejemplo, este código no funcionará:

```
// ...
article.createTodays(); /// Error: article.creat
```

Propiedades estáticas

Una adición reciente

Esta es una adición reciente al lenguaje. Los ejemplos funcionan en el Chrome reciente.

Las propiedades estáticas también son posibles, se ven como propiedades de clase regular, pero precedidas por `static`:

```
class Article {
  static publisher = "Ilya Kantor";
}

alert( Article.publisher ); // Ilya Kantor
```

Eso es lo mismo que una asignación directa a `Article`:

```
Article.publisher = "Ilya Kantor";
```

Herencia de propiedades y métodos estáticos

Las propiedades y métodos estáticos son heredados.

Por ejemplo, `Animal.compare` y `Animal.planet` en el siguiente código son heredados y accesibles como `Rabbit.compare` y `Rabbit.planet`:

```
class Animal {
  static planet = "Tierra";
  constructor(name, speed) {
    this.speed = speed;
    this.name = name;
  }

  run(speed = 0) {
    this.speed += speed;
```

```
    alert(`${this.name} corre a una velocidad de ${thi
  }

  static compare(animalA, animalB) {
    return animalA.speed - animalB.speed;
  }

}

// Hereda de Animal
class Rabbit extends Animal {
  hide() {
    alert(`${this.name} se esconde!`);
  }
}

let rabbits = [
  new Rabbit("Conejo Blanco", 10),
  new Rabbit("Conejo Negro", 5)
];

rabbits.sort(Rabbit.compare);

rabbits[0].run(); // Conejo Negro corre a una velocida
```

Ahora, cuando llamemos a `Rabbit.compare`, se llamará a `Animal.compare` heredado.

¿Como funciona? Nuevamente, usando prototipos. Como ya habrás adivinado, `extends` da a `Rabbit` el `[[Prototype]]` referente a `Animal`.

Entonces, `Rabbit extends Animal` crea dos referencias `[[Prototype]]`:

1. La función de `Rabbit` se hereda prototípicamente de la función de `Animal`.
2. `Rabbit.prototype` prototípicamente hereda de `Animal.prototype`.

Como resultado, la herencia funciona tanto para métodos regulares como estáticos.

Verifiquemos eso por código, aquí:

```
class Animal {}
class Rabbit extends Animal {}

// para la estática
alert(Rabbit.__proto__ === Animal); // verdadero

// para métodos regulares
alert(Rabbit.prototype.__proto__ === Animal.prototype)
```

Resumen

Los métodos estáticos se utilizan en la funcionalidad propia de la clase "en su conjunto". No se relaciona con una instancia de clase concreta.

Por ejemplo, un método para comparar `Article.compare (article1, article2)` o un método de fábrica `Article.createTodays()`.

Están etiquetados por la palabra `static` en la declaración de clase.

Las propiedades estáticas se utilizan cuando queremos almacenar datos a nivel de clase, también no vinculados a una instancia.

La sintaxis es:

```
class MyClass {
  static property = ...;

  static method() {
    ...
  }
}
```

Técnicamente, la declaración estática es lo mismo que asignar a la clase misma:

```
MyClass.property = ...
MyClass.method = ...
```

Las propiedades y métodos estáticos se heredan.

Para `class B extends A` el prototipo de la clase `B` en sí mismo apunta a `A`: `B.[[Prototipo]] = A`. Entonces, si no se encuentra un campo en `B`, la búsqueda continúa en `A`.

Tareas

¿La clase extiende el objeto?

importancia: 3

Como sabemos, todos los objetos normalmente heredan de `Object.prototype` y obtienen acceso a métodos de objeto "genéricos" como `hasOwnProperty` etc.

Por ejemplo:

```
class Rabbit {
  constructor(name) {
    this.name = name;
  }
}

let rabbit = new Rabbit("Rab");

// el método hasOwnProperty proviene de Object.prototy
alert( rabbit.hasOwnProperty('name') ); // verdadero
```

Pero si lo escribimos explícitamente como `"class Rabbit extends Object"`, entonces ¿el resultado sería diferente de una simple `"class Rabbit"`?

¿Cuál es la diferencia?

Aquí un ejemplo de dicho código (no funciona – ¿por qué? ¿Arréglalo?):

```
class Rabbit extends Object {
  constructor(name) {
    this.name = name;
  }
}

let rabbit = new Rabbit("Rab");

alert( rabbit.hasOwnProperty('name') ); // Error
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

Propiedades y métodos privados y protegidos.

Uno de los principios más importantes de la programación orientada a objetos: delimitar la interfaz interna de la externa.

Esa es una práctica "imprescindible" en el desarrollo de algo más complejo que una aplicación "hola mundo".

Para entender esto, alejémonos del desarrollo y volvamos nuestros ojos al mundo real...

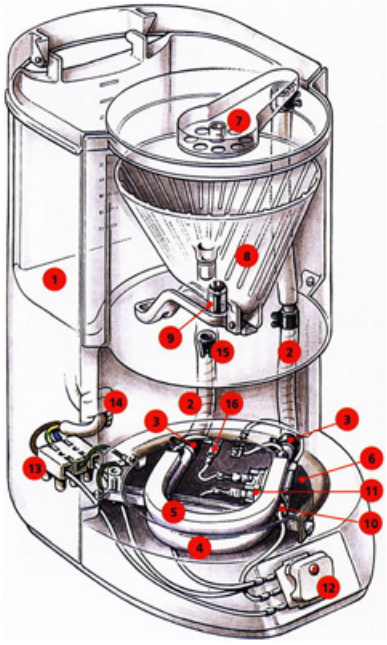
Por lo general, los dispositivos que estamos usando son bastante complejos. Pero delimitar la interfaz interna de la externa permite usarlas sin problemas.

Un ejemplo de la vida real

Por ejemplo, una máquina de café. Simple desde el exterior: un botón, una pantalla, algunos agujeros ... Y, seguramente, el resultado: ¡excelente café! :)



Pero adentro ... (una imagen del manual de reparación)



Muchos detalles. Pero podemos usarlo sin saber nada.

Las cafeteras son bastante confiables, ¿no es así? Podemos usarlos por años, y solo si algo sale mal, tráigalo para repararlo.

El secreto de la fiabilidad y la simplicidad de una máquina de café: todos los detalles están bien ajustados y *ocultos* en su interior.

Si retiramos la cubierta protectora de la cafetera, su uso será mucho más complejo (¿dónde presionar?) Y peligroso (puedes electrocutarte).

Como veremos, en la programación los objetos son como máquinas de café.

Pero para ocultar detalles internos, no utilizaremos una cubierta protectora, sino una sintaxis especial del lenguaje y las convenciones.

Interfaz interna y externa

En la programación orientada a objetos, las propiedades y los métodos se dividen en dos grupos:

- *Interfaz interna* – métodos y propiedades, accesibles desde otros métodos de la clase, pero no desde el exterior.
- *Interfaz externa* – métodos y propiedades, accesibles también desde fuera de la clase.

Si continuamos la analogía con la máquina de café, lo que está oculto en su interior: un tubo de caldera, un elemento calefactor, etc., es su interfaz interna.

Se utiliza una interfaz interna para que el objeto funcione, sus detalles se utilizan entre sí. Por ejemplo, un tubo de caldera está unido al elemento calefactor.

Pero desde afuera, una máquina de café está cerrada por la cubierta protectora, para que nadie pueda alcanzarlos. Los detalles están ocultos e inaccesibles. Podemos usar sus funciones a través de la interfaz externa.

Entonces, todo lo que necesitamos para usar un objeto es conocer su interfaz externa. Es posible que no seamos completamente conscientes de cómo funciona dentro, y eso es genial.

Esa fue una introducción general.

En JavaScript, hay dos tipos de campos de objeto (propiedades y métodos):

- Público: accesible desde cualquier lugar. Comprenden la interfaz externa. Hasta ahora solo estábamos usando propiedades y métodos públicos.
- Privado: accesible solo desde dentro de la clase. Estos son para la interfaz interna.

En muchos otros lenguajes también existen campos "protegidos": accesibles solo desde dentro de la clase y aquellos que lo extienden (como privado, pero más acceso desde clases heredadas). También son útiles para la interfaz interna. En cierto sentido, están más extendidos que los privados, porque generalmente queremos que las clases heredadas tengan acceso a ellas.

Los campos protegidos no se implementan en JavaScript a nivel de lenguaje, pero en la práctica son muy convenientes, por lo que se emulan.

Ahora haremos una máquina de café en JavaScript con todos estos tipos de propiedades. Una máquina de café tiene muchos detalles, no los modelaremos todos, seremos simples (aunque podríamos).

Proteger "waterAmount"

Hagamos primero una clase de cafetera simple:

```
class CoffeeMachine {
  waterAmount = 0; // la cantidad de agua adentro

  constructor(power) {
    this.power = power;
    alert( `Se creó una máquina de café, poder: ${powe
  }

}

// se crea la máquina de café
let coffeeMachine = new CoffeeMachine(100);

// agregar agua
coffeeMachine.waterAmount = 200;
```

En este momento las propiedades `waterAmount` y `power` son públicas. Podemos obtenerlos/configurarlos fácilmente desde el exterior a cualquier valor.

Cambiemos la propiedad `waterAmount` a protegida para tener más control sobre ella. Por ejemplo, no queremos que nadie lo ponga por debajo de cero.

Las propiedades protegidas generalmente tienen el prefijo de subrayado `_`.

Eso no se aplica a nivel de lenguaje, pero existe una convención bien conocida entre los programadores de que no se debe acceder a tales propiedades y métodos desde el exterior.

Entonces nuestra propiedad se llamará `_waterAmount`:

```
class CoffeeMachine {
  _waterAmount = 0;

  set waterAmount(value) {
    if (value < 0) {
```

```
      value = 0;
    }
    this._waterAmount = value;
  }

  get waterAmount() {
    return this._waterAmount;
  }

  constructor(power) {
    this._power = power;
  }

}

// se crea la máquina de café
let coffeeMachine = new CoffeeMachine(100);

// agregar agua
```

Ahora el acceso está bajo control, por lo que establecer una cantidad de agua por debajo de cero se volvió imposible.

"Power" de solo lectura

Para la propiedad `power`, hagámoslo de solo lectura. A veces sucede que una propiedad debe establecerse solo en el momento de la creación y nunca modificarse.

Ese es exactamente el caso de una máquina de café: la potencia nunca cambia.

Para hacerlo, solo necesitamos hacer `getter`, pero no `setter`:

```
class CoffeeMachine {
  // ...

  constructor(power) {
    this._power = power;
  }

  get power() {
    return this._power;
  }

}

// se crea la máquina de café
let coffeeMachine = new CoffeeMachine(100);

alert(`La potencia es: ${coffeeMachine.power}W`); // P

coffeeMachine.power = 25; // Error (sin setter)
```

Funciones getter/setter

Aquí usamos la sintaxis getter/setter.

Pero la mayoría de las veces las funciones `get.../set...` son preferidas, como esta:

```
class CoffeeMachine {
  _waterAmount = 0;

  setWaterAmount(value) {
    if (value < 0) value = 0;
    this._waterAmount = value;
  }

  getWaterAmount() {
    return this._waterAmount;
  }
}

new CoffeeMachine().setWaterAmount(100);
```

Eso parece un poco más largo, pero las funciones son más flexibles. Pueden aceptar múltiples argumentos (incluso si no los necesitamos en este momento).

Por otro lado, la sintaxis get/set es más corta, por lo que, en última instancia, no existe una regla estricta, depende de usted decidir.

Los campos protegidos son heredados.

Si heredamos `class MegaMachine extends CoffeeMachine`, entonces nada nos impide acceder a `this._waterAmount` o `this._power` desde los métodos de la nueva clase.

Por lo tanto, los campos protegidos son naturalmente heredables. A diferencia de los privados que veremos a continuación.

"#waterLimit" Privada

Una adición reciente

Esta es una adición reciente al lenguaje. No es compatible con motores de JavaScript, o es compatible parcialmente todavía, requiere polyfilling.

Hay una propuesta de JavaScript terminada, casi en el estándar, que proporciona soporte a nivel de lenguaje para propiedades y métodos privados.

Los privados deberían comenzar con `#`. Solo son accesibles desde dentro de la clase.

Por ejemplo, aquí hay una propiedad privada `#waterLimit` y el método privado de control de agua `#fixWaterAmount`:

```
class CoffeeMachine {
  #waterLimit = 200;

  #fixWaterAmount(value) {
    if (value < 0) return 0;
    if (value > this.#waterLimit) return this.#waterLi
  }

  setWaterAmount(value) {
    this.#waterLimit = this.#fixWaterAmount(value);
  }

}

let coffeeMachine = new CoffeeMachine();

// no puede acceder a privados desde fuera de la clase
coffeeMachine.#fixWaterAmount(123); // Error
coffeeMachine.#waterLimit = 1000; // Error
```

A nivel de lenguaje, `#` es una señal especial de que el campo es privado. No podemos acceder desde fuera o desde clases heredadas.

Los campos privados no entran en conflicto con los públicos. Podemos tener campos privados `#waterAmount` y públicos `waterAmount` al mismo tiempo.

Por ejemplo, hagamos que `waterAmount` sea un accesorio para `#waterAmount`:

```
class CoffeeMachine {

  #waterAmount = 0;

  get waterAmount() {
    return this.#waterAmount;
  }

  set waterAmount(value) {
    if (value < 0) value = 0;
    this.#waterAmount = value;
  }
}

let machine = new CoffeeMachine();

machine.waterAmount = 100;
alert(machine.#waterAmount); // Error
```

A diferencia de los protegidos, los campos privados son aplicados por el propio lenguaje. Eso es bueno.

Pero si heredamos de `CoffeeMachine`, entonces no tendremos acceso directo a `#waterAmount`. Tendremos que confiar en el getter/setter de `waterAmount`:

```
class MegaCoffeeMachine extends CoffeeMachine {
  method() {
```

```
    alert( this.#waterAmount ); // Error: solo se pued
  }
}
```

En muchos escenarios, esta limitación es demasiado severa. Si ampliamos una `CoffeeMachine`, es posible que tengamos razones legítimas para acceder a sus componentes internos. Es por eso que los campos protegidos se usan con más frecuencia, aunque no sean compatibles con la sintaxis del lenguaje.

⚠ Los campos privados no están disponibles como this[name]

Los campos privados son especiales.

Como sabemos, generalmente podemos acceder a los campos usando `this[name]`:

```
class User {
  ...
  sayHi() {
    let fieldName = "nombre";
    alert(`Hello, ${this[fieldName]}`);
  }
}
```

Con campos privados eso es imposible: `this['#name']` no funciona. Esa es una limitación de sintaxis para garantizar la privacidad.

Resumen

En términos de POO, la delimitación de la interfaz interna de la externa se llama encapsulamiento.

Ofrece los siguientes beneficios:

Protección para los usuarios, para que no se disparen en el pie

Imagínese, hay un equipo de desarrolladores que usan una máquina de café. Fue hecho por la compañía "Best CoffeeMachine" y funciona bien, pero se quitó una cubierta protectora. Entonces la interfaz interna está expuesta.

Todos los desarrolladores son civilizados: usan la máquina de café según lo previsto. Pero uno de ellos, John, decidió que él era el más inteligente e hizo algunos ajustes en el interior de la máquina de café. Entonces la máquina de café falló dos días después.

Seguramente no es culpa de John, sino de la persona que quitó la cubierta protectora y dejó que John hiciera sus manipulaciones.

Lo mismo en programación. Si un usuario de una clase cambiará cosas que no están destinadas a ser cambiadas desde el exterior, las consecuencias son impredecibles.

Soportable

La situación en la programación es más compleja que con una máquina de café de la vida real, porque no solo la compramos una vez. El código se somete

constantemente a desarrollo y mejora.

Si delimitamos estrictamente la interfaz interna, el desarrollador de la clase puede cambiar libremente sus propiedades y métodos internos, incluso sin informar a los usuarios.

Si usted es un desarrollador de tal clase, es bueno saber que los métodos privados se pueden renombrar de forma segura, sus parámetros se pueden cambiar e incluso eliminar, porque ningún código externo depende de ellos.

Para los usuarios, cuando sale una nueva versión, puede ser una revisión total internamente, pero aún así es simple de actualizar si la interfaz externa es la misma.

Ocultando complejidad

La gente adora usar cosas que son simples. Al menos desde afuera. Lo que hay dentro es algo diferente.

Los programadores no son una excepción.

Siempre es conveniente cuando los detalles de implementación están ocultos, y hay disponible una interfaz externa simple y bien documentada.

Para ocultar una interfaz interna utilizamos propiedades protegidas o privadas:

- Los campos protegidos comienzan con `_`. Esa es una convención bien conocida, no aplicada a nivel de lenguaje. Los programadores solo deben acceder a un campo que comience con `_` de su clase y las clases que hereden de él.
- Los campos privados comienzan con `#`. JavaScript se asegura de que solo podamos acceder a los que están dentro de la clase.

En este momento, los campos privados no son compatibles entre los navegadores, pero se puede usar "polyfill".

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

 17 de noviembre de 2025

Ampliación de clases nativas

Las clases nativas como Array, Map y otras también son extensibles.

Por ejemplo, aquí `PowerArray` hereda del `Array` nativo:

```
// se agrega un método más (puedes hacer más)
class PowerArray extends Array {
  isEmpty() {
    return this.length === 0;
  }
}

let arr = new PowerArray(1, 2, 5, 10, 50);
alert(arr.isEmpty()); // falso

let filteredArr = arr.filter(item => item >= 10);
alert(filteredArr); // 10, 50
alert(filteredArr.isEmpty()); // falso
```

Tenga en cuenta una cosa muy interesante. Métodos nativos como `filter`, `map`, y otros, devuelven los nuevos objetos exactamente del tipo heredado `PowerArray`. Su implementación interna utiliza la propiedad `constructor` del objeto para eso.

En el ejemplo anterior,

```
arr.constructor === PowerArray
```

Cuando se llama a `arr.filter()`, crea internamente la nueva matriz de resultados usando exactamente `arr.constructor`, no el básico `Array`. En realidad, eso es muy bueno, porque podemos seguir usando métodos `PowerArray` más adelante en el resultado.

Aún más, podemos personalizar ese comportamiento.

Podemos agregar un `getter` estático especial `Symbol.species` a la clase. Si existe, debería devolver el constructor que JavaScript usará internamente para crear nuevas entidades en `map`, `filter` y así sucesivamente.

Si queremos que los métodos nativos como `map` o `filter` devuelvan matrices regulares, podemos devolver `Array` en `Symbol.species`, como aquí:

```
class PowerArray extends Array {
  isEmpty() {
    return this.length === 0;
  }


```

```
  // los métodos nativos usarán esto como el construct
  static get [Symbol.species]() {
    return Array;
  }
}

let arr = new PowerArray(1, 2, 5, 10, 50);
alert(arr.isEmpty()); // falso

// filter crea una nueva matriz usando arr.constructor
let filteredArr = arr.filter(item => item >= 10);

// filterArr no es PowerArray, sino Array
alert(filteredArr.isEmpty()); // Error: filteredArr.is
```

Como puede ver, ahora `.filter` devuelve un `Array`. Por lo tanto, la funcionalidad extendida ya no se pasa.

> **Otras colecciones también trabajan del mismo modo**
>
> Otras colecciones, como `Map` y `Set`, funcionan igual. También usan `Symbol.species`.

Sin herencia estática en nativos

Los objetos nativos tienen sus propios métodos estáticos, por ejemplo, `Object.keys`, `Array.isArray`, etc.

Como ya sabemos, las clases nativas se extienden entre sí. Por ejemplo, `Array` extiende `Object`.

Normalmente, cuando una clase extiende a otra, se heredan los métodos estáticos y no estáticos. Eso se explicó a fondo en el artículo Propiedades y métodos estáticos..

Pero las clases nativas son una excepción. No heredan estáticos el uno del otro.

Por ejemplo, tanto `Array` como `Date` heredan de `Object`, por lo que sus instancias tienen métodos de `Object.prototype`. Pero `Array.[[Prototype]]` no hace referencia a `Object`, por lo que no existe, por ejemplo, el método estático `Array.keys()` (o `Date.keys()`).

Imagen de la estructura para `Date` y `Object`:

Como puede ver, no hay un vínculo entre `Date` y `Object`. Son independientes, solo `Date.prototype` hereda de `Object.prototype`.

Esa es una diferencia importante de herencia entre los objetos nativos en comparación con lo que obtenemos con 'extends`.

Comentarios

lea esto antes de comentar...



© 2007—2026 Ilya Kantor

Comprobación de clase: "instanceof"

El operador `instanceof` permite verificar si un objeto pertenece a una clase determinada. También tiene en cuenta la herencia.

Tal verificación puede ser necesaria en muchos casos. Aquí lo usaremos para construir una función *polimórfica*, la que trata los argumentos de manera diferente dependiendo de su tipo.

El operador instanceof

La sintaxis es:

```
obj instanceof Class
```

Devuelve `true` si `obj` pertenece a `Class` o una clase que hereda de ella.

Por ejemplo:

```
class Rabbit {}
let rabbit = new Rabbit();

// ¿Es un objeto de la clase Rabbit?
alert( rabbit instanceof Rabbit ); // true
```

También funciona con funciones de constructor:

```
// en lugar de clase
function Rabbit() {}

alert( new Rabbit() instanceof Rabbit ); // true
```

...Y con clases nativas como `Array`:

```
let arr = [1, 2, 3];
alert( arr instanceof Array ); // true
alert( arr instanceof Object ); // true
```

Tenga en cuenta que `arr` también pertenece a la clase `Object`. Esto se debe a que `Array` hereda prototípicamente de `Object`.

Normalmente, `instanceof` examina la cadena de prototipos para la verificación. También podemos establecer una lógica personalizada en el método estático `Symbol.hasInstance`.

El algoritmo de `obj instanceof Class` funciona más o menos de la siguiente manera:

1. Si hay un método estático `Symbol.hasInstance`, simplemente llámelo: `Class[Symbol.hasInstance](obj)`. Debería devolver `true` o `false`, y hemos terminado. Así es como podemos personalizar el comportamiento de `instanceof`.

   Por ejemplo:

```
// Configurar la verificación con `instanceof`, que
// cualquier objeto que tenga la propiedad `canEat`

class Animal {
  static [Symbol.hasInstance](obj) {
    if (obj.canEat) return true;
  }
}

let obj = { canEat: true };

alert(obj instanceof Animal); // true: Animal[Symbol
```

2. La mayoría de las clases no tienen `Symbol.hasInstance`. En ese caso, se utiliza la lógica estándar: `obj instanceof Class` comprueba si `Class.prototype` es igual a alguno de los prototipos en la cadena de prototipos de `obj`.

   En otras palabras, compara uno tras otro:

```
obj.__proto__ === Class.prototype?
obj.__proto__.__proto__ === Class.prototype?
obj.__proto__.__proto__.__proto__ === Class.prototyp
...
// si alguna respuesta es verdadera, devuelve true
// de lo contrario, si llegamos al final de la cader
```

   En el ejemplo anterior `rabbit.__proto__ === Rabbit.prototype`, por lo que da la respuesta de inmediato.

   En el caso de una herencia, la coincidencia será en el segundo paso:

```
class Animal {}
class Rabbit extends Animal {}

let rabbit = new Rabbit();
alert(rabbit instanceof Animal); // true

// rabbit.__proto__ === Animal.prototype (no match)
// rabbit.__proto__.__proto__ === Animal.prototype (
```

Aquí está la ilustración de lo que `rabbit instanceof Animal` compara con `Animal.prototype`:

Por cierto, también hay un método objA.isPrototypeOf(objB), que devuelve `true` si `objA` está en algún lugar de la cadena de prototipos para `objB`. Por lo tanto, la prueba de `obj instanceof Class` se puede reformular como `Class.prototype.isPrototypeOf(obj)`.

Es extraño, ¡pero el constructor `Class` en sí mismo no participa en el chequeo! Solo importa la cadena de prototipos y `Class.prototype`.

Eso puede llevar a consecuencias interesantes cuando se cambia una propiedad `prototype` después de crear el objeto.

Como aquí:

```
function Rabbit() {}
let rabbit = new Rabbit();

// cambió el prototipo
Rabbit.prototype = {};

// ...ya no es un conejo!
alert( rabbit instanceof Rabbit ); // false
```

Bonificación: Object.prototype.toString para el tipo

Ya sabemos que los objetos simples se convierten en cadenas como `[objetc Objetc]`:

```
let obj = {};

alert(obj); // [object Object]
alert(obj.toString()); // lo mismo
```

Esa es su implementación de `toString`. Pero hay una característica oculta que hace que `toString` sea mucho más poderoso que eso. Podemos usarlo como

un `typeof` extendido y una alternativa para `instanceof`.

¿Suena extraño? En efecto. Vamos a desmitificar.

Por esta especificación, el `toString` incorporado puede extraerse del objeto y ejecutarse en el contexto de cualquier otro valor. Y su resultado depende de ese valor.

- Para un número, será `[object Number]`
- Para un booleano, será `[objetc Boolean]`
- Para `null`: `[objetc Null]`
- Para `undefined`: `[objetc Undefined]`
- Para matrices: `[Object Array]`
- ... etc (personalizable).

Demostremos:

```
// copie el método toString en una variable a conveni
let objectToString = Object.prototype.toString;

// ¿que tipo es este?
let arr = [];

alert( objectToString.call(arr) ); // [object Array]
```

Aquí usamos call como se describe en el capítulo Decoradores y redirecciones, call/apply para ejecutar la función `objectToString` en el contexto `this=arr`.

Internamente, el algoritmo `toString` examina `this` y devuelve el resultado correspondiente. Más ejemplos:

```
let s = Object.prototype.toString;

alert( s.call(123) ); // [object Number]
alert( s.call(null) ); // [object Null]
alert( s.call(alert) ); // [object Function]
```

Symbol.toStringTag

El comportamiento del objeto `toString` se puede personalizar utilizando una propiedad de objeto especial `Symbol.toStringTag`.

Por ejemplo:

```
let user = {
  [Symbol.toStringTag]: "User"
};

alert( {}.toString.call(user) ); // [object User]
```

Para la mayoría de los objetos específicos del entorno, existe dicha propiedad. Aquí hay algunos ejemplos específicos del navegador:

```
// toStringTag para el objeto y clase específicos del
alert( window[Symbol.toStringTag]); // ventana
alert( XMLHttpRequest.prototype[Symbol.toStringTag] );

alert( {}.toString.call(window) ); // [object Window]
alert( {}.toString.call(new XMLHttpRequest()) ); // [o
```

Como puedes ver, el resultado es exactamente `Symbol.toStringTag` (si existe), envuelto en `[object ...]`.

Al final tenemos "typeof con esteroides" que no solo funciona para tipos de datos primitivos, sino también para objetos incorporados e incluso puede personalizarse.

Podemos usar `{}.toString.call` en lugar de `instanceof` para los objetos nativos cuando deseamos obtener el tipo como una cadena en lugar de solo verificar.

Resumen

Resumamos los métodos de verificación de tipos que conocemos:

	trabaja para	retorna
`typeof`	primitivos	cadena
`{}.toString`	primitivos, objetos incorporados, objetos con `Symbol.toStringTag`	cadena
`instanceof`	objetos	true/false

Como podemos ver, `{}.toString` es técnicamente un `typeof` "más avanzado".

Y el operador `instanceof` realmente brilla cuando estamos trabajando con una jerarquía de clases y queremos verificar si la clase tiene en cuenta la herencia.

Tareas

Extraño instanceof

importancia: 5

En el siguiente código, ¿por qué `instanceof` devuelve `true`? Podemos ver fácilmente que `a` no es creado por `B()`.

```
function A() {}
function B() {}

A.prototype = B.prototype = {};

let a = new A();

alert( a instanceof B ); // verdadero
```

solución

Comentarios

lea esto antes de comentar...

© 2007—2026 Ilya Kantor

Los Mixins

En JavaScript podemos heredar de un solo objeto. Solo puede haber un `[[Prototype]]` para un objeto. Y una clase puede extender únicamente otra clase.

Pero a veces eso se siente restrictivo. Por ejemplo, tenemos una clase `StreetSweeper` y una clase `Bicycle`, y queremos hacer su combinación: un `StreetSweepingBicycle`.

O tenemos una clase `User` y una clase `EventEmitter` que implementa la generación de eventos, y nos gustaría agregar la funcionalidad de `EventEmitter` a `User`, para que nuestros usuarios puedan emitir eventos.

Hay un concepto que puede ayudar aquí, llamado "mixins".

Como se define en Wikipedia, un mixin es una clase que contiene métodos que pueden ser utilizados por otras clases sin necesidad de heredar de ella.

En otras palabras, un *mixin* proporciona métodos que implementan cierto comportamiento, pero su uso no es exclusivo, lo usamos para agregar el comportamiento a otras clases.

Un ejemplo de mixin

La forma más sencilla de implementar un mixin en JavaScript es hacer un objeto con métodos útiles, para que podamos combinarlos fácilmente en un prototipo de cualquier clase.

Por ejemplo, aquí el mixin `sayHiMixin` se usa para agregar algo de "diálogo" a `User`:

```
// mixin
let sayHiMixin = {
  sayHi() {
    alert(`Hola ${this.name}`);
  },
  sayBye() {
    alert(`Adiós ${this.name}`);
  }
};

// uso:
class User {
  constructor(name) {
    this.name = name;
  }
}

// copia los métodos
Object.assign(User.prototype, sayHiMixin);
```

```
// Ahora el User puede decir hola
new User("tío").sayHi(); // Hola tío!
```

No hay herencia, sino un simple método de copia. Entonces, `User` puede heredar de otra clase y también incluir el mixin para "mezclar" los métodos adicionales, como este:

```
class User extends Person {
  // ...
}

Object.assign(User.prototype, sayHiMixin);
```

Los mixins pueden hacer uso de la herencia dentro de sí mismos.

Por ejemplo, aquí `sayHiMixin` hereda de `sayMixin`:

```
let sayMixin = {
  say(phrase) {
    alert(phrase);
  }
};

let sayHiMixin = {
  __proto__: sayMixin, // (o podríamos usar Object.set

  sayHi() {
    // llama al método padre
    super.say(`Hola ${this.name}`); // (*)
  },
  sayBye() {
    super.say(`Adios ${this.name}`); // (*)
  }
};

class User {
  constructor(name) {
    this.name = name;
  }
}

// copia los métodos
Object.assign(User.prototype, sayHiMixin);

// User ahora puede decir hola
new User("tío").sayHi(); // Hola tío!
```

Ten en cuenta que la llamada al método padre `super.say()` de `sayHiMixin` (en las líneas etiquetadas con `(*)`) busca el método en el prototipo de ese mixin, no en la clase.

Aquí está el diagrama (ver la parte derecha):

Esto se debe a que los métodos `sayHi` y `sayBye` se crearon inicialmente en `sayHiMixin`. Entonces, a pesar de que se copiaron, su propiedad interna `[[[HomeObject]]]` hace referencia a `sayHiMixin`, como se muestra en la imagen de arriba.

Como `super` busca los métodos padres en `[[HomeObject]].[[Prototype]]`, esto significa que busca `sayHiMixin.[[Prototype]]`.

EventMixin

Ahora hagamos un mixin para la vida real.

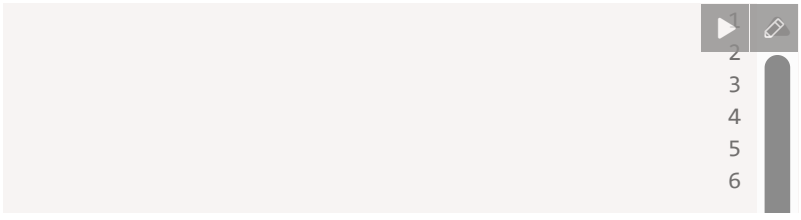
Una característica importante de muchos objetos del navegador (por ejemplo) es que pueden generar eventos. Los eventos son una excelente manera de "transmitir información" a cualquiera que lo desee. Así que hagamos un mixin que nos permita agregar fácilmente funciones relacionadas con eventos a cualquier clase/objeto.

- El mixin proporcionará un método `.trigger(name, [...data])` para "generar un evento" cuando le ocurra algo importante. El argumento `name` es un nombre del evento, opcionalmente seguido de argumentos adicionales con datos del evento.
- También el método `.on(name, handler)` que agrega la función `handler` como listener a eventos con el nombre dado. Se llamará cuando se desencadene un evento con el nombre `name` dado, y obtenga los argumentos de la llamada `.trigger`.
- ...Y el método `.off(name, handler)` que elimina el listener `handler`.

Después de agregar el mixin, un objeto `user` podrá generar un evento `"login"` cuando el visitante inicie sesión. Y otro objeto, por ejemplo, `calendar` puede querer escuchar dichos eventos para cargar el calendario para el persona registrada.

O bien, un `menu` puede generar el evento `"seleccionar"` cuando se selecciona un elemento del menú, y otros objetos pueden asignar controladores para reaccionar ante ese evento. Y así.

Aquí está el código:



```
let eventMixin = {
 /**
  * Suscribe al evento, uso:
  *  menu.on('select', function(item) { ... }
 */
 on(eventName, handler) {
   if (!this._eventHandlers) this._eventHandlers = {};
   if (!this._eventHandlers[eventName]) {
     this._eventHandlers[eventName] = [];
   }
   this._eventHandlers[eventName].push(handler);
 },

 /**
  * Cancelar la suscripción, uso:
  *  menu.off('select', handler)
  */
 off(eventName, handler) {
   let handlers = this._eventHandlers?.[eventName];
   if (!handlers) return;
   for (let i = 0; i < handlers.length; i++) {
     if (handlers[i] === handler) {
       handlers.splice(i--, 1);
     }
   }
 },
```

```
/**
 * Generar un evento con el nombre y los datos
 *  this.trigger('select', data1, data2);
 */
trigger(eventName, ...args) {
 if (!this._eventHandlers?.[eventName]) {
    return; // no hay controladores para ese nombre de
  }

  // Llama al controlador
  this._eventHandlers[eventName].forEach(handler => han
}
```

- `.on(eventName, handler)`: asigna la función `handler` para que se ejecute cuando se produce el evento con ese nombre. Técnicamente, hay una propiedad `_eventHandlers` que almacena una matriz de controladores para cada nombre de evento, y simplemente la agrega a la lista.
- `.off(eventName, handler)` – elimina la función de la lista de controladores.
- `.trigger(eventName, ...args)` – genera el evento: se llama a todos los controladores de `_eventHandlers[eventName]`, con una lista de argumentos `...args`.

Uso:

```
1  // Construir una clase
2  class Menu {
3    choose(value) {
4      this.trigger("select", value);
5    }
6  }
7  // Agrega el mixin con métodos relacionados con evento
8  Object.assign(Menu.prototype, eventMixin);
9
10 let menu = new Menu();
11
12 // agrega un controlador, que se llamará en la selecci
13 menu.on("select", value => alert(`Valor seleccionado:
14
15 // desencadena el evento => el controlador anterior se
16 // Valor seleccionado: 123
17 menu.choose("123");
```

Ahora, si queremos que el código reaccione a una selección de menú, podemos escucharlo con `menu.on(...)`.

Y el mixin de `eventMixin` hace que sea fácil agregar ese comportamiento a tantas clases como queramos, sin interferir con la cadena de herencia.

Resumen

Mixin – es un término genérico de programación orientado a objetos: una clase que contiene métodos para otras clases.

Algunos lenguajes permiten la herencia múltiple. JavaScript no admite la herencia múltiple, pero los mixins se pueden implementar copiando métodos en el prototipo.

Podemos usar mixins como una forma de expandir una clase agregando múltiples comportamientos, como el manejo de eventos que hemos visto anteriormente.

Los mixins pueden convertirse en un punto de conflicto si sobrescriben accidentalmente los métodos de clase existentes. Por lo tanto, generalmente debes planificar correctamente la definición de métodos de un mixin, para minimizar la probabilidad de que suceda.

Comentarios

lea esto antes de comentar...



© 2007—2026 Ilya Kantor